

Daily Study Notes

Python File Handling • SQL Window Functions

■ Python — File Handling

Opening Files

```
with open("employees.txt", "r") as file:
    content = file.read()
```

Modes

Mode	Meaning
"r"	Read
"w"	Write / Overwrite
"a"	Append

Reading

Method	Description
file.read()	Reads the entire file
file.readline()	Reads one line
file.readlines()	Reads all lines as a list

Line-by-Line Processing

```
with open("employees.txt", "r") as input_file:
    while True:
        content = input_file.readline()

        if not content:
            break
```

```
        # process content
```

Important: `content.strip()` removes whitespace/newline characters.

```
name, skill, salary = content.strip().split(",")
salary = int(salary)
```

Writing

```
with open("high_salary.txt", "w") as output_file:
    output_file.write("Alice,Python,100000\n")
```

Appending

```
with open("high_salary.txt", "a") as output_file:  
    output_file.write("Eva,Pandas,85000\n")
```

Practical Pipeline

Input file → read → strip → split → convert → filter → write output

■ You completed practical file handling today.

■ SQL — Window Functions

1. Window Function Concept

The biggest difference: **GROUP BY** combines rows into a single summarized row, while a **WINDOW FUNCTION** calculates across rows but keeps every individual row in the result.

```
SELECT name,
       departmentId,
       salary,
       AVG(salary) OVER (
           PARTITION BY departmentId
       ) AS department_avg
FROM Employee_2;
```

Each employee remains in the result while their department average is shown alongside them.

2. PARTITION BY

PARTITION BY departmentId means: perform the calculation separately for each department.

```
IT
|-- Alice
|-- Bob
\-- Charlie

HR
|-- David
\-- Eva
```

3. ROW_NUMBER()

```
ROW_NUMBER() OVER (
    PARTITION BY departmentId
    ORDER BY salary DESC
)
```

Use it when you need unique sequential numbering.

```
SELECT name,
       departmentId,
       salary,
       ROW_NUMBER() OVER (
           PARTITION BY departmentId
           ORDER BY salary DESC
       ) AS row_num
FROM Employee_2;
```

4. RANK()

```
RANK() OVER (
    PARTITION BY departmentId
    ORDER BY salary DESC
)
```

Same salaries receive the same rank, but gaps occur:

Salary	Rank
100000	1
100000	1
90000	3

5. DENSE_RANK()

```
DENSE_RANK() OVER (
    PARTITION BY departmentId
    ORDER BY salary DESC
)
```

Same salaries receive the same rank, but no gaps:

Salary	Dense Rank
100000	1
100000	1
90000	2

Remember

Function	Duplicate Values	Gap
ROW_NUMBER()	Different number	—
RANK()	Same rank	Yes
DENSE_RANK()	Same rank	No

6. Top N Per Department ■

This is a very important interview pattern.

For the top 2 employees from each department:

```
SELECT name,
       departmentId,
       salary,
       row_num
FROM (
    SELECT name,
           departmentId,
           salary,
           ROW_NUMBER() OVER (
               PARTITION BY departmentId
               ORDER BY salary DESC
           ) AS row_num
    FROM Employee_2
```

```
) AS ranked
WHERE row_num <= 2;
```

Pattern: Inner query → calculate ROW_NUMBER() → Outer query → filter row_num

WHERE row_num = 1 → highest-paid employee from each department

WHERE row_num <= 3 → top 3 from each department

7. LIMIT and OFFSET

LIMIT restricts the number of returned rows.

```
SELECT name, salary
FROM Employee_2
ORDER BY salary DESC
LIMIT 3;
```

→ Top 3 employees overall.

For LeetCode 176 (Second Highest Salary):

```
SELECT (
    SELECT DISTINCT salary
    FROM Employee
    ORDER BY salary DESC
    LIMIT 1 OFFSET 1
) AS SecondHighestSalary;
```

LIMIT → number of rows to take | **OFFSET** → number of rows to skip

Important Distinction

Tool	Use Case
LIMIT	Top N overall
ROW_NUMBER() + PARTITION BY	Top N from EACH group

■ What You Should Remember for Interviews

The most important patterns from today:

```
AVG(...) OVER (PARTITION BY ...)
```

-> Group average while keeping rows

```
ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ... DESC)
```

-> Ranking within groups / Top N per group

```
RANK() OVER (...)
```

-> Ranking with ties and gaps

```
DENSE_RANK() OVER (...)
```

-> Ranking with ties, no gaps

```
LIMIT + OFFSET
```

-> Select a particular position from an ordered result